Day 171: Early Stopping

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 4 | Complete study guide

Watch: L53: Early stopping: preventing overfitting with validation monitoring (IIT Madras) | Early Stopping in Keras to Prevent Overfitting (Jeff Heaton)

Script: `day171_early_stopping.py` · every number in this guide is copied by program from `day171_run_output.txt`, the real output of that script.

Early stopping watches a held-out stop set after every epoch, stops once its loss has not improved for a while, and goes back to the best epoch's weights. On UCI Adult with Day 170's network it was close to free: with a patience of 5 epochs, training on 2,000 rows stopped after 17.8 epochs on average instead of 100, and the kept weights scored 0.3418 on fresh rows, against 0.5254 for the 100th epoch and 0.3416 for the best epoch in hindsight. Keeping the last weights instead of the best cost more as the patience grew. The real danger was a small stop set: with 100 rows it chose worse epochs and its own score at the chosen epoch was +0.0150 too good.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

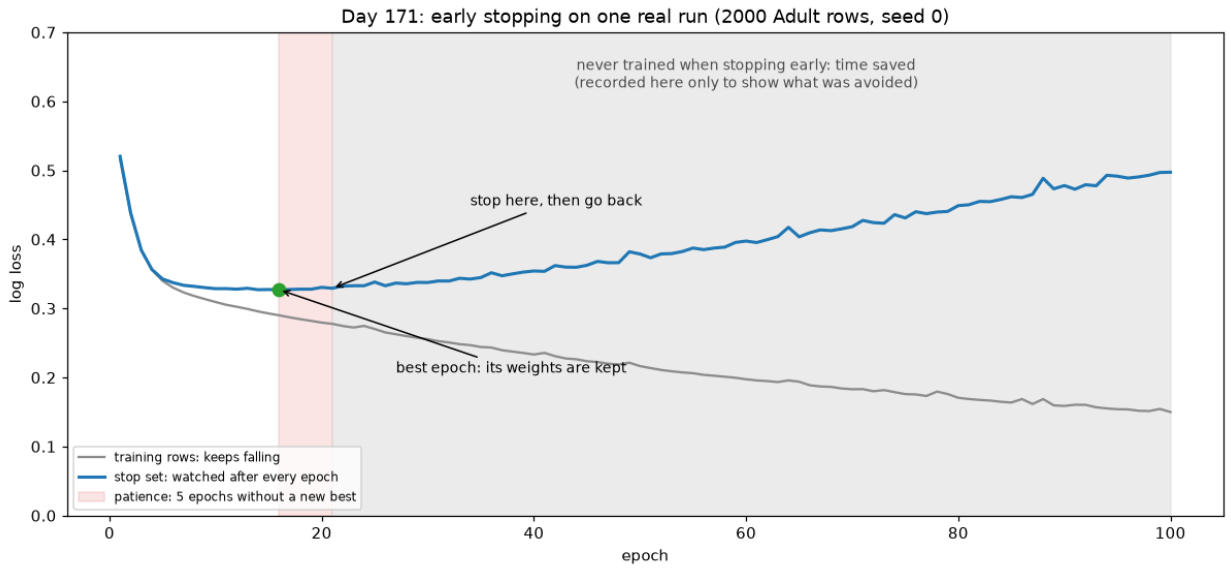
Over-fitting: after a while, a network starts to memorise its training rows. The loss on them keeps falling, while the loss on rows it has not seen turns and rises.

Early stopping keeps a separate **stop set**, scores it after every epoch (one pass over the training rows), and stops once the score has not improved for **patience** epochs. It then **restores** the weights from the best epoch. A third set of **fresh rows** is kept to score the result, because the stop set has been used to choose.

REAL-WORLD SCENARIO

Studying for an exam with past papers: at first every paper improves your score on new ones; later you are only memorising the answers to those papers. A friend who quizzes you on unseen questions each evening tells you when to stop, and you would want to remember how you knew things on your best evening.

SYSTEM DIAGRAM



One real run (2,000 Adult rows, seed 0): the stop set's loss is lowest at epoch 16; after 5 epochs without a new best training stops at 21 and the epoch-16 weights are restored.

Data: Week 24's split of UCI Adult. Of the 24,130 fitting rows, 6,000 are the stop set and 2,000 or 18,000 of the rest train; the 6,032 validation rows are the fresh rows, which score every choice and choose nothing. The test file is not touched. The network is Day 170's ($87 \to 64 \to 32 \to 1$, Adam at rate 10^{-3}, inside Day 170's good range). Every run is recorded for all 100 epochs, so every rule can be replayed on the same curves; section 5 checks that replaying gives exactly what running the stopper for real gives.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. What over-fitting looks like

DEFINITION — start here, no prior knowledge assumed

Ten seeds at each training size, 100 epochs each, scoring the training rows, the stop set and the fresh rows after every epoch.

A control: the 2,000 training rows with their labels shuffled, so there is nothing to learn but noise. The best possible model then predicts the base rate for everyone, whose log loss can be worked out exactly.

REAL-WORLD SCENARIO

A student who has memorised a wrong answer key: they get better at the key and worse at the subject.

SPECIFICATION

```
sizes: 2,000 and 18,000 training rows; 10 seeds each; 100 epochs; batches of 128
control: 2,000 rows, labels shuffled; base rate q, base log loss = -mean[y log q + (1 - y) log(1 - q)]
```

IMPLEMENTATION

```
def trajectory(D, n, seed, shuffle=False):
    Xa, ya = D["X"][D["train"][:n]], D["y"][D["train"][:n]]
    if shuffle:
        ya = ya[np.random.RandomState(99 + seed).permutation(n)]
    sizes = [Xa.shape[1], *d170.HIDDEN, 1]; acts = ["relu"] * len(d170.HIDDEN) + ["linear"]
    net = d126.init_net(sizes, seed); opt = d126.Adam(net, LR); rng = np.random.RandomState(seed +
        1)
    tr, st, rows, fr = [], [], [], []
    t0 = time.time()
    for e in range(MAX_EPOCHS):
        train_epoch(net, opt, acts, Xa, ya, rng)
        tr.append(row_losses(net, acts, Xa, ya).mean())
        ls = row_losses(net, acts, D["X"][D["stop"]], D["y"][D["stop"]])
        st.append(ls.mean()); rows.append(ls.astype(np.float32))
        fr.append(row_losses(net, acts, D["X"][D["fresh"]], D["y"][D["fresh"]]).mean())
    return dict(train=np.array(tr), stop=np.array(st), rows=np.array(rows), fresh=np.array(fr),
        sec=time.time() - t0)
```

ACTUAL OUTPUT

```
   2000 training rows: best epoch on the stop set: median 13 (range 10 to 16); on the fresh rows: median 12
        training loss at epoch 100 0.1517; fresh loss: at the stop set's best epoch 0.3418, at epoch 100 0.5254, best in
            hindsight 0.3416
  18000 training rows: best epoch on the stop set: median 12 (range 6 to 14); on the fresh rows: median 7
        training loss at epoch 100 0.2211; fresh loss: at the stop set's best epoch 0.3298, at epoch 100 0.4333, best in
            hindsight 0.3287
  control, 2000 rows with shuffled labels: best epoch on the stop set: median 9; fresh loss there 0.5669, at epoch 100
      0.8872; always predicting the base rate 0.247: 0.5611
```

What it means

Over-fitting is fast and large. With 2,000 rows the stop set's best epoch had a median of 13 (range 10 to 16); by epoch 100 the training loss had fallen to 0.1517 while the fresh loss had risen to 0.5254, against 0.3418 at the stop set's best epoch. With 18,000 rows the rise was smaller (0.4333 against 0.3298) but still large.

The stop set and the fresh rows mostly agree. At 2,000 rows the fresh rows' best epoch had a median of 12 (on the fresh rows: median 12); at 18,000 rows the stop set preferred later epochs (median 12) than the fresh rows (median 7), but the curves are so flat near the bottom that this cost only about 0.001.

The control behaves as it should. With shuffled labels, early stopping kept weights scoring 0.5669, close to the 0.5611 of always predicting the base rate, while 100 epochs reached 0.8872: worse than knowing nothing.

WHY CHOOSE THIS?

One stop set turns a 100-epoch guess into a choice made from data, and it caught pure memorisation.

WHY NOT THIS? (TRADEOFFS)

It needs rows held back from training and a score after every epoch.

2. Stopping rules: what each costs and saves

DEFINITION — start here, no prior knowledge assumed

Each rule is replayed on the 20 recorded curves. Patience p: stop after p epochs without a new best. Prechelt's generalisation-loss rule (GL, 1998): stop once the loss is more than α percent above its best so far. Both restore the best epoch seen.

Regret is the fresh loss of the kept weights minus the best fresh loss of any epoch, which no rule can know in advance. Two baselines: train all 100 epochs and keep the last weights, or keep the best of all 100.

REAL-WORLD SCENARIO

Deciding when to leave a queue: wait for one more minute of no movement, or five, or until the line is 5% longer than at its shortest.

MATHEMATICS

$$\textit{patience: stop at the first } t \textit{ with } t - t^* \geq p, \textit{ where } t^* = \operatorname{argmin}_{s \leq t} E(s); \textit{ keep the weights from } t^*$$

$$GL(t) = 100\,(E(t) / \min_{s \leq t} E(s) - 1); \textit{ stop at the first } t \textit{ with } GL(t) > \alpha$$

IMPLEMENTATION

```
class EarlyStopper:
    def __init__(self, patience=5, min_delta=0.0):
        self.patience, self.min_delta = patience, min_delta
        self.best, self.best_epoch, self.best_net, self.wait = np.inf, -1, None, 0
    def update(self, epoch, score, net=None):
        if score < self.best - self.min_delta:
```

```
            self.best, self.best_epoch, self.wait = score, epoch, 0
            self.best_net = copy_net(net) if net is not None else None
        else:
            self.wait += 1
        return self.wait >= self.patience
def stop_at(curve, patience, min_delta=0.0):
    es = EarlyStopper(patience, min_delta)
    for e, v in enumerate(curve):
        if es.update(e, v):
            return e, es.best_epoch
    return len(curve) - 1, es.best_epoch
def gl_stop_at(curve, alpha):
    best = np.minimum.accumulate(np.asarray(curve, float))
    gl = 100 * (np.asarray(curve, float) / best - 1)
    hit = np.flatnonzero(gl > alpha)
    e = int(hit[0]) if len(hit) else len(curve) - 1
    return e, int(np.argmin(curve[:e + 1]))
```

ACTUAL OUTPUT

```
  2000 training rows                epochs run   fresh loss   regret (mean / worst)   kept the overall best
    patience 1                      11.2       0.3421   +0.0006 / +0.0021   0.3
    patience 2                      14.5       0.3419   +0.0004 / +0.0014   0.9
    patience 5                      17.8       0.3418   +0.0003 / +0.0014   1.0
    patience 10                     22.8       0.3418   +0.0003 / +0.0014   1.0
    patience 20                     32.8       0.3418   +0.0003 / +0.0014   1.0
    GL > 1%                         19.8       0.3418   +0.0003 / +0.0014   1.0
    GL > 2%                         24.6       0.3418   +0.0003 / +0.0014   1.0
    GL > 5%                         34.2       0.3418   +0.0003 / +0.0014   1.0
    no stopping, best              100.0       0.3418   +0.0003 / +0.0014   1.0
    no stopping, last              100.0       0.5254   +0.1839 / +0.2219   0.0
  18000 training rows                epochs run   fresh loss   regret (mean / worst)   kept the overall best
    patience 1                       4.8       0.3312   +0.0025 / +0.0085   0.0
    patience 2                       8.8       0.3292   +0.0005 / +0.0012   0.2
    patience 5                      15.9       0.3298   +0.0011 / +0.0023   1.0
    patience 10                     20.9       0.3298   +0.0011 / +0.0023   1.0
    patience 20                     30.9       0.3298   +0.0011 / +0.0023   1.0
    GL > 1%                         13.4       0.3298   +0.0012 / +0.0023   0.8
    GL > 2%                         20.9       0.3298   +0.0011 / +0.0023   1.0
    GL > 5%                         32.6       0.3298   +0.0011 / +0.0023   1.0
    no stopping, best              100.0       0.3298   +0.0011 / +0.0023   1.0
    no stopping, last              100.0       0.4333   +0.1046 / +0.1276   0.0
  (regret: the fresh loss of the kept weights minus the best fresh loss of any epoch, known only in hindsight;
   kept the overall best: the share of runs whose kept epoch is the stop set's best of all 100)
```

What it means

Patience 5 lost almost nothing and saved most of the work. At 2,000 rows it ran 17.8 epochs and kept the stop set's best of all 100 in every run (share 1.0); at 18,000 rows, 15.9 epochs. Longer patience and the GL rules gave the same weights at a higher cost.

Patience 1 was too impatient at 18,000 rows: 4.8 epochs, never the overall best, and the worst run +0.0085 behind. At 2,000 rows it also stopped before the overall best in most runs (share 0.3 kept it), but the curve is so flat near its bottom that this cost only +0.0006.

A surprise: at 18,000 rows patience 2 (+0.0005) beat patience 5 (+0.0011). It is not a better rule: it stopped before the stop set's own best, which here happened to sit later than the fresh rows' best (section 1). With 10 seeds and differences of 0.0006, it should not be read as a recommendation.

Not stopping at all and keeping the last weights cost +0.1839 and +0.1046.

WHY CHOOSE THIS?

A patience of about 5 epochs gave the best weights here at a fifth of the cost.

WHY NOT THIS? (TRADEOFFS)

These curves are smooth and U-shaped; a noisy curve or a slow learner (a plateau before a second drop) needs more patience, and this lesson did not test one.

3. Restore the best weights, or keep the last ones?

DEFINITION — start here, no prior knowledge assumed

The same rules, but this time comparing the weights at the best epoch with the weights at the epoch where training stopped, p epochs later.

REAL-WORLD SCENARIO

Coming home from a long walk: you want the photo from the summit, not the last one you took on the way down.

ACTUAL OUTPUT

```
   2000 rows, fresh loss restored / last: p1: 0.3421 / 0.3427, p2: 0.3419 / 0.3429, p5: 0.3418 / 0.3442, p10: 0.3418 /
       0.3479, p20: 0.3418 / 0.3622
  18000 rows, fresh loss restored / last: p1: 0.3312 / 0.3323, p2: 0.3292 / 0.3310, p5: 0.3298 / 0.3316, p10: 0.3298 /
      0.3362, p20: 0.3298 / 0.3439
```

What it means

Restoring always helped, and more as the patience grew: at 2,000 rows from p1: 0.3421 / 0.3427 to p20: 0.3418 / 0.3622; at 18,000 rows up to p20: 0.3298 / 0.3439. Twenty epochs past the best, the network has over-fitted again. Restoring costs one copy of the weights each time the score improves.

WHY CHOOSE THIS?

Restoring makes a long patience safe: waiting longer can only find a better stop-set epoch, never a worse one.

WHY NOT THIS? (TRADEOFFS)

It needs a copy of the weights in memory or on disk, which for a large model is not free.

4. How big must the stop set be?

DEFINITION — start here, no prior knowledge assumed

Random draws split the stop set into two disjoint halves A and B of the same size (100, 500 or 3,000 rows). Patience 5 stops and chooses on A only. Regret is measured on the fresh rows as before.

Optimism: B's loss minus A's loss at the epoch A chose. A chose the epoch where it looked best, so its score there is flattered. **Control**: the same halves at an epoch chosen on the full stop set, which A did not choose alone; it should be about zero. The paired difference removes the luck of which rows fell in A. 10 runs × 100 draws per size.

REAL-WORLD SCENARIO

Judging a school by its best day out of twenty: the best day is partly luck, and on a small class it is mostly luck.

MATHEMATICS

$\hat{e} = \operatorname{argmin}_t L_A(t)$ *(with patience 5)* $\quad optimism = L_B(\hat{e}) - L_A(\hat{e}) \quad control = L_B(f) - L_A(f)$, *f chosen without A alone*

ACTUAL OUTPUT

```
   2000 rows, halves of  100: regret +0.0064; optimism +0.0159, control +0.0009, difference +0.0150 (standard error
       0.00063); epochs run 19.8
   2000 rows, halves of  500: regret +0.0019; optimism +0.0038, control -0.0000, difference +0.0038 (standard error
       0.00017); epochs run 19.0
   2000 rows, halves of 3000: regret +0.0005; optimism +0.0004, control -0.0004, difference +0.0008 (standard error
       0.00004); epochs run 18.5
  18000 rows, halves of  100: regret +0.0044; optimism +0.0162, control +0.0007, difference +0.0155 (standard error
      0.00060); epochs run 13.9
  18000 rows, halves of  500: regret +0.0025; optimism +0.0069, control +0.0004, difference +0.0064 (standard error
      0.00023); epochs run 14.6
  18000 rows, halves of 3000: regret +0.0013; optimism +0.0013, control -0.0004, difference +0.0016 (standard error
      0.00007); epochs run 15.6
  (optimism: B's loss minus A's at the epoch A chose; control: the same halves at an epoch chosen on the full stop set;
   difference: optimism minus control, draw by draw, which removes the luck of which rows fell in A)
```

What it means

Small stop sets choose worse. With 100 rows the regret was +0.0064 (2,000 training rows), against +0.0005 with 3,000.

And they flatter themselves. With 100 rows A's score at its chosen epoch was +0.0150 too good (standard error 0.00063); with 3,000 rows only +0.0008. The controls stayed near zero (+0.0009, -0.0000, -0.0004 at 2,000 rows), so the optimism comes from choosing, not from the halves. This is Day 168's winner's curse again: **never report the stop set's score as the model's score**; that is what the fresh rows are for.

WHY CHOOSE THIS?

A few thousand stop rows made the choice reliable and its score honest.

WHY NOT THIS? (TRADEOFFS)

Every row in the stop set is a row the model does not train on.

5. The stopper run for real, and what it saves

DEFINITION — start here, no prior knowledge assumed

Sections 2 to 4 replay rules on recorded curves. Here `train_es` trains with an `EarlyStopper(5)` for real, and is compared with the rule replayed on seed 0's recorded curve. Timing: the same loop with a patience of 101, which never stops.

REAL-WORLD SCENARIO

Checking a flight simulator against one real flight before trusting it for the rest.

IMPLEMENTATION

```
def train_es(Xa, ya, Xs, ys, opt_cls, lr, seed, stopper, max_epochs=MAX_EPOCHS, batch=128):
    sizes = [Xa.shape[1], *d170.HIDDEN, 1]; acts = ["relu"] * len(d170.HIDDEN) + ["linear"]
    net = d126.init_net(sizes, seed); opt = opt_cls(net, lr); rng = np.random.RandomState(seed +
        1)
    curve = []
    for e in range(max_epochs):
        train_epoch(net, opt, acts, Xa, ya, rng, batch)
        curve.append(row_losses(net, acts, Xs, ys).mean())
        if stopper.update(e, curve[-1], net):
            break
    return stopper.best_net, acts, np.array(curve)
```

ACTUAL OUTPUT

```
   2000 rows: run for real, stopped after 21 epochs, best epoch 16, fresh loss 0.3399; replayed: stopped after 21, best 16,
       fresh 0.3399; curves identical: True
         time: 0.16 s, against 0.71 s for the same loop with patience 101 (all 100 epochs)
  18000 rows: run for real, stopped after 17 epochs, best epoch 12, fresh loss 0.3289; replayed: stopped after 17, best 12,
      fresh 0.3289; curves identical: True
         time: 0.76 s, against 4.14 s for the same loop with patience 101 (all 100 epochs)
```

What it means

Running for real and replaying agree exactly: both stopped after 21 epochs at 2,000 rows with best epoch 16, and the stop-set losses were identical epoch by epoch (curves identical: True). That is why replaying recorded curves in sections 2 to 4 was valid: the training is deterministic, and a rule only decides where to cut it.

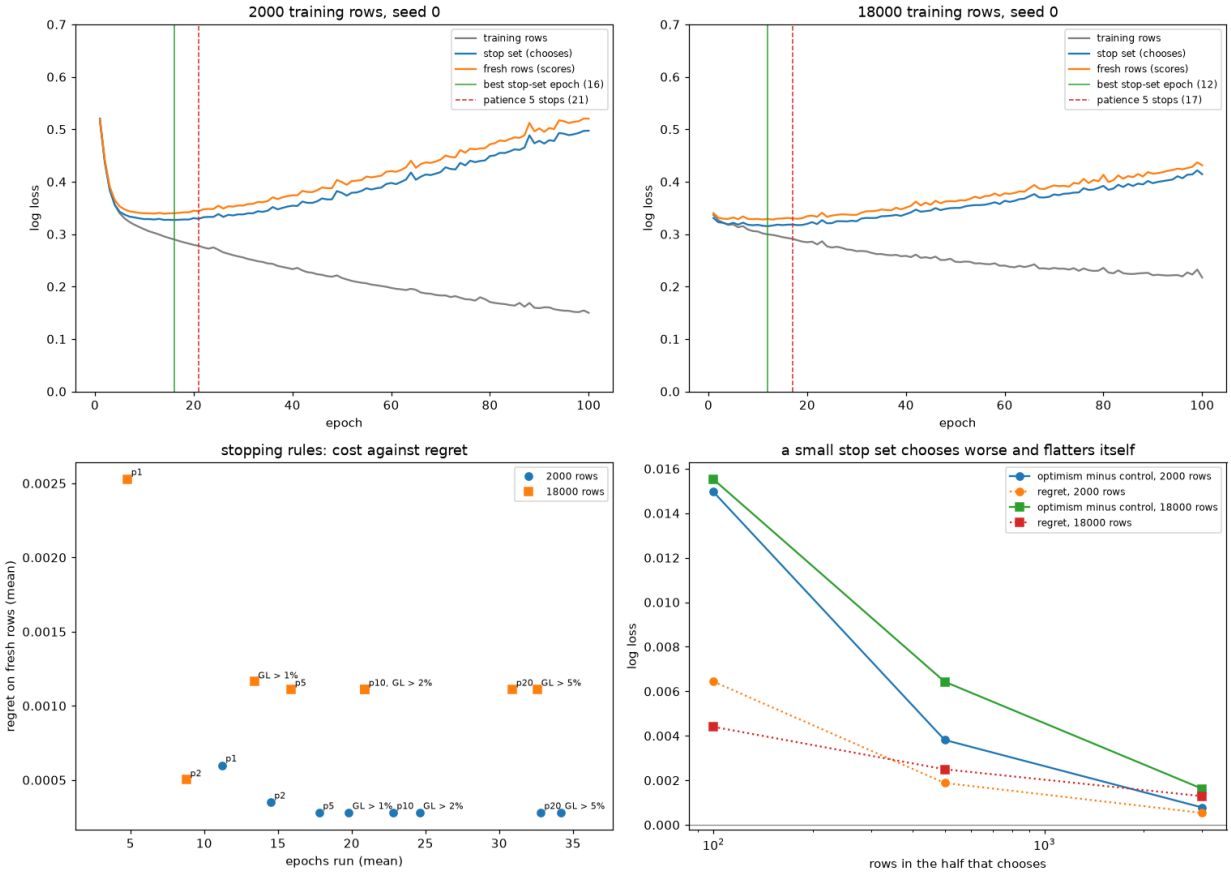
Time: 0.76 s against 4.14 s at 18,000 rows. (Timings on this machine swing between runs; the epochs saved are the steadier measure.)

Today's part for the week's build

`day171_exercise.py` holds `copy_net`, `EarlyStopper`, `stop_at`, `gl_stop_at`, `row_losses`, `train_epoch` and `train_es`, copied unchanged. Its toy check first used rate 10^{-2}, where the best epoch was the first and the restore check proved little; at 10^{-3} the network learns for several epochs first. Its hand checks:

ACTUAL OUTPUT

```
copy_net: original weight now 9, copy 1 (by hand 9 and 1)
stop_at([5, 4, 4.5, 4.2, 4.1, 3.9], patience 2): (3, 1) (by hand (3, 1): best at epoch 1, then
    epochs 2 and 3 are no better, so it stops at 3 and never sees 3.9)
stop_at([1.0, 0.995, 0.99, 0.985], patience 2, min_delta 0.01): (2, 0) (by hand (2, 0): neither
    0.995 nor 0.99 is 0.01 better than 1.0)
gl_stop_at([1.0, 0.8, 0.82, 0.85], alpha 5): (3, 1) (by hand (3, 1): 0.82 is 2.5% above the best
    0.8, 0.85 is 6.25%)
row_losses at logit 0, labels 1 and 0: [0.6931, 0.6931] (by hand ln 2 = 0.6931 for both)
toy problem (200 training rows, 20% noisy labels): stopped after 15 of 300 epochs, best epoch 10;
    restored weights score 0.581452, the curve's lowest 0.581452 (must be equal; no model can beat
    the noise floor -0.2 ln 0.2 - 0.8 ln 0.8 = 0.5004)
```



Top: one run at each training size: training, stop-set and fresh loss, the best stop-set epoch (green) and where patience 5 stops (red). Bottom left: each rule's epochs run against its regret. Bottom right: optimism beyond the control, and regret, against the size of the half that chooses.

Interview preparation — technical questions

Q: Why restore the best weights instead of keeping the last?

A: Patience epochs past the best the network is over-fitting again; here 20 epochs past cost 0.02 at 2,000 rows.

Q: How do you choose the patience?

A: Long enough to ride out wobbles, short enough to save time; with restoring, longer only costs time. Here 5 found the best epoch in every run.

Q: Can you report the stop set's loss as the model's score?

A: No: it chose the epoch, so it is flattered (0.015 with 100 rows); score on rows that chose nothing.

Q: How do you check an early-stopping implementation?

A: Replay the rule on a recorded curve and compare, and check the restored weights reproduce the lowest loss.

Q: What does early stopping cost?

A: A held-out set, a score every epoch, and copies of the weights.

Interview preparation — theoretical questions

Q: Why is early stopping a regulariser?

A: It limits how far the weights travel from their starting point, much like a penalty on their size.

Q: Why does the stop set's choice flatter its own score?

A: The minimum of noisy estimates is biased low: the winner's curse of Day 168.

Q: When can a small patience fail?

A: When the loss wobbles or plateaus before falling again; patience 1 stopped at 4.8 epochs at 18,000 rows.

Q: Early stopping or more data?

A: Both help; with 18,000 rows over-fitting was slower and smaller, but it still happened.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Early stopping is close to free when the curve is smooth: a patience of about five epochs with the best weights restored found the best epoch and saved most of the training. Its real requirements are a stop set of a few thousand rows and a separate set to report the score.

- Patience 5, 2,000 rows: 17.8 epochs, fresh 0.3418 against 0.5254 at epoch 100.
- Keeping the last weights with patience 20: 0.3622 against 0.3418 restored.
- A 100-row stop set: regret +0.0064, and its own score +0.0150 too good.
- Shuffled labels: early stopping kept 0.5669, near the base rate's 0.5611.

Suggested watch order, if starting from zero

- 1. L53: Early stopping: preventing overfitting with validation monitoring (IIT Madras) — a lecture on early stopping as validation monitoring.
- 2. Early Stopping in Keras to Prevent Overfitting (Jeff Heaton) — early stopping and restoring the best weights in practice.

Day 172 preview

- **Cross-validation for time series and grouped data.** Shuffled folds let the model see the future, or the same user in training and test; a real leakage demo on CO2, electrical equipment and MovieLens.